Retrieval-Augmented Generation(RAG)

A detailed whitepaper by Disha Bhavsar, Mayank Tiwari

What is RAG and why use it? **3**
Main modules of RAG: **6**
Data preparation pipeline: 6
Document ingestion 6
Chunking 7
Embedding generation 10
Vector database 12
Query processing pipeline: 15
Query embeddings and vector similarity searches 15
From word embeddings to query embedding: 16
Similarity Searches: 18
Context construction pipeline: 19
Context packaging: 19
Prompt template construction 20
Decomposing: 20
Generation pipeline: 22
Post-processing: 23
Evaluation and improvement: 26
Retrieval Evaluation: 27
Retrieval improvement: 27
Generative evaluation: 28
Applications and Advantages: **28**
Code implementation: **29**
References: **33**

What is RAG and why use it?

Retrieval-Augmented Generation (RAG) is an architectural pattern for Large Language Models (LLMs) designed to significantly enhance their capacity for generating responses that are both accurate and contextually pertinent. This enhancement is achieved by grounding the generation process in external, verifiable knowledge sources.

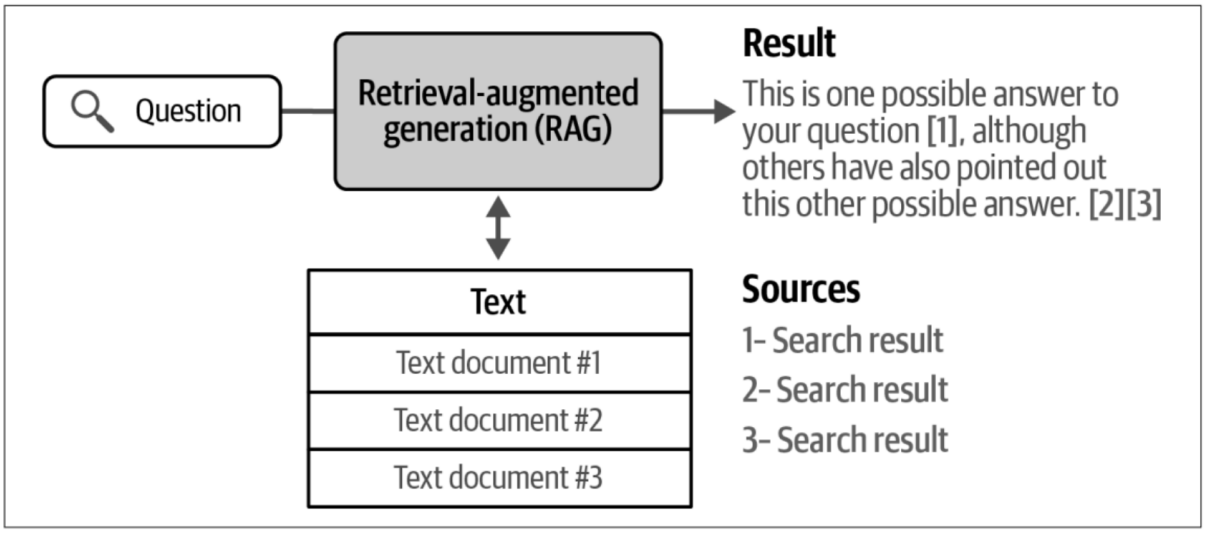


Figure: *The RAG system formulates a response to a query by incorporating information from relevant, available documents.*

RAG integrates an information retrieval system with a generative LLM in three conceptual phases:

1. **Retrieval:** When a user submits a query, the system searches an external knowledge base (often stored in a Vector Database) to find the most relevant document passages or chunks that relate to the query.

2. **Augmentation:** These retrieved passages are dynamically inserted into the prompt given to the LLM. The original user query is thus *augmented* with factual context.
3. **Generation:** The LLM uses this rich, augmented prompt to generate an accurate, grounded, and context-specific response, relying on the injected data rather than solely its internal, pre-trained parameters.

LLMs are typically trained on vast amounts of public data up to a specific cutoff date. This leads to two critical limitations that RAG solves:

1. **Knowledge Gap:** LLMs cannot answer questions about events, documents, or data that emerged after their training completion. Foundation models have broad knowledge, but can lack depth in specialized domains.
2. **Hallucination:** When an LLM lacks confidence or internal knowledge for a query, it often generates plausible-sounding but factually incorrect information. Models are trained on a diverse set of data that can contain contradictions, errors, and ambiguous data (in addition to the correct data). Because of this, models assign probabilities to all possible continuations, even the wrong ones.The result is output that can contain hallucinations.

As fine-tuning an LLM is a highly costly operation, RAG is a compelling strategy that bypasses the need for constant fine-tuning to access that new data. By enabling dynamic retrieval and incorporation of external knowledge during inference, RAG systems transform static language models into adaptive, knowledge-aware systems. This capability has proven essential for applications requiring up-to-date information,

domain-specific knowledge, or factual grounding that extends beyond pre-training corpora.

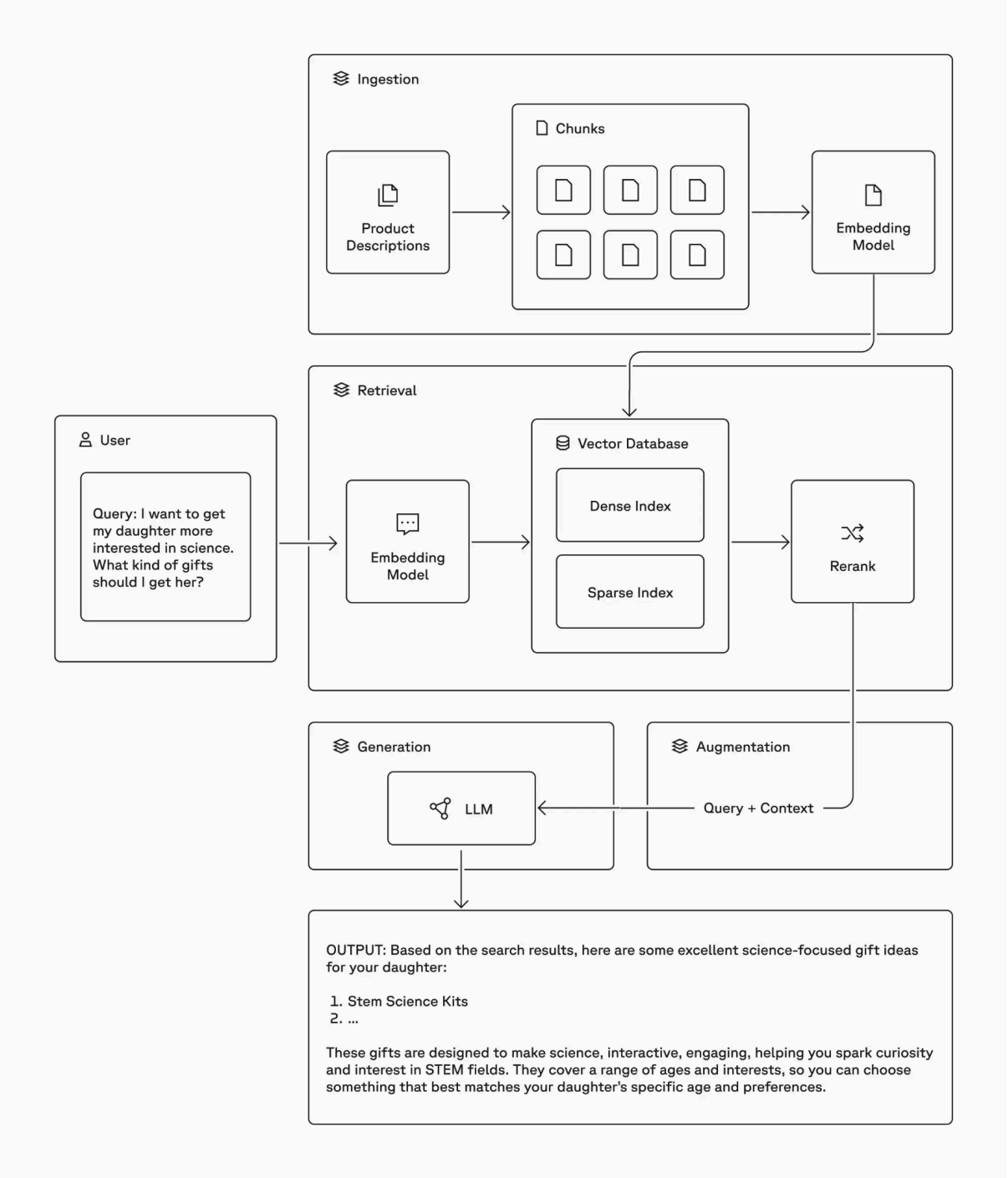


Figure: *Traditional RAG*

RAG model can be represented mathematically as:

$$P(y|x)=\sum P_{\Theta}(y|x,d)P_{R}(d|x)$$

where:

- ➔ x: user query
- ➔ d: retrieved document chunks
- ➔ P_R: retrieval distribution
- ➔ P_Θ: generation model

Main modules of RAG:

Data preparation pipeline:

Document ingestion

It is the crucial first phase of feeding external data into a Retrieval-Augmented Generation system. Ingestion is the process of parsing information from source documents so that it can be embedded into a search space for later retrieval.
While this is a straightforward process for plain text, complications arise when the source documents are in non 'text' formats, eg. Microsoft Word or PDF, and when they contain complex formatting such as repeating headers and footers, text in multiple columns, or tables.

Its basic steps include:
Loading: Gathering documents from diverse sources (files, databases, web).

Parsing/Extraction: Converting different formats (PDF, HTML, Word) into plain text.

Chunking

Chunking in Retrieval Augmented Generation (RAG) is the process of splitting large documents into smaller, semantically meaningful segments or "chunks".

There are two big reasons why chunking is necessary for any application involving vector databases or LLMs:

1. It ensures embedding models can fit the data into their context windows.
2. It ensures the chunks themselves contain the information necessary for search.

Common chunking strategies include:

1. **<u>Fixed-size chunking</u>**:
   This is the most crude and simplest method of segmenting the text. It breaks down the text into chunks of a specified number of characters, regardless of their content or structure.
   - Advantages: Simplicity, Efficiency, Consistency, Low computational requirements
   - Disadvantages: Lacks structural/content density adaptability, Potential information loss, Sub-optimal for heterogeneous content, Context fragmentation

2. **<u>Recursive chunking</u>**:
   In this method, we divide the text into smaller chunks in a hierarchical and iterative manner using a set of separators (“\n\n”, “\n”, “ “,””). It tries each separator (like paragraphs, sentences, or specific markers) in a descending order of importance to find the most meaningful boundaries in the text.

If the initial attempt at splitting the text doesn't produce chunks of the desired size, the method recursively calls itself on the resulting chunks with a different separator until the desired chunk size is achieved.

- Advantages: Preservation of structural integrity, Fine-grained control, Handles complex content
- Disadvantages: Dependence on separators (ineffective if the text lacks clear separators), Slower performance due to higher computational overhead

3. **<u>Document based chunking</u>** :
   It splits the document based on its inherent structures. This method aims to preserve the full structure and context of the document, making it ideal for content where splitting may disrupt the flow or meaning. It ensures that key information and context remain intact across the document.
   - Advantages: Ideal for structured texts, Full context preservation
   - Disadvantages: Limited specificity, Scalability issues, Documentation being resource-intensive

4. **<u>Semantic chunking</u>**:
   It breaks text into chunks based on meaning rather than fixed sizes. It ensures that each chunk contains coherent and relevant information by analyzing shifts in the text's semantic structure. This is typically done by measuring differences in sentence embeddings (mathematical representation).
   This approach aims to improve retrieval accuracy by ensuring each chunk contains a single, focused meaning, which leads to better performance for downstream tasks.

$$d_i = \arg\min_d \sum_{s \in d} \| f(s) - \mu_d \|^2$$

: where f is an embedding function.

- Advantages: Preserves meaning, Adaptable to diverse content, Improves retrieval accuracy
- Disadvantages: Complex setup, Threshold tuning, Higher computational cost (analyzing meaning differences especially for longer documents)

5. **<u>Sentence based chunking</u>**:
   Token-based chunking in RAG involves splitting text into segments based on a predetermined number of tokens, which can be words or subwords. This method is highly effective for models with specific token processing limits, as it directly aligns with their input constraints.
   - Advantages:Optimized for language models (like GPT), Precise control over chunk size, Consistency over content
   - Disadvantages:Context fragmentation, Ignores semantic structure, ignorance of content density

6. **<u>Token-based chunking</u>**:

   Sentence-based chunking divides text into full sentences, ensuring that each chunk contains complete thoughts. This method helps preserve the logical flow of information by splitting natural sentence boundaries.
   - Advantages:Context preservation, Better readability, Natural division (splitting at logical points)
   - Disadvantages:Variable chunk sizes, Inefficient for long sentences (exceeding token limit or information overload might occur)

7. **<u>Agentic chunking</u>**:
   It is an advanced text segmentation technique that makes use of AI-based text-splitting method and leverages LLMs to break down documents into semantically meaningful

pieces.The hypothesis here is to process documents in a fashion that humans would do.
Meaningful context is preserved in all the smaller chunks, making this approach incredibly important to chatbots, knowledge bases, and generative ai (gen ai) use cases.

- Advantages:Task-oriented efficiency (tailored chunks for specific tasks or actions), Better focus on relevant data
- Disadvantages:Over-specialization, Loss of global context (affects quality of holistic tasks like summarization)

8. **Hierarchical chunking**:
   It is an advanced type of chunking strategy which breaks content into multiple layers of structure, making it easier to analyze or retrieve information at different levels of detail. This nested approach preserves context across levels and is especially useful for documents with natural structure, such as articles, reports, documentation, or long-form text.
   - Advantages:Preservation of multi-level structure (paragraphs, sentences, etc.), allows hierarchical analysis
   - Disadvantages: Boundary detection errors (inconsistent or missing separators)

Each chunk can be enriched with metadata and converted into numerical representations (embeddings) for improved semantic understanding.
Overlapping segments maintain context between chunks, boosting the likelihood of retrieving relevant information and improving system performance.

Embedding generation

This is the process where we convert text and other data into high dimensional numerical vectors which can be stored in vector databases. The purpose of embeddings is to capture the

semantic meaning of the text, such that words with similar meaning are located closer in the vector space.
In RAG, embeddings are used to retrieve relevant documents from a large corpus based on the input query (given by user).

Mechanism:

- The entire database is processed in order to create embeddings for each document or chunks of texts. This essentially maps the entire corpus into high-dimensional vector space.

$$f : d_i \rightarrow v_i \in R^n$$

- When the user inputs a query, it itself is converted into an embedding. This query embedding represents the semantic meaning of the user's input in the same vector space as the corpus.
- The system then compares the query embedding with all the document embeddings in the vector space. It calculates similarity scores and retrieves the most similar documents.
- The retrieved documents are then used as additional context or knowledge to generate a response.

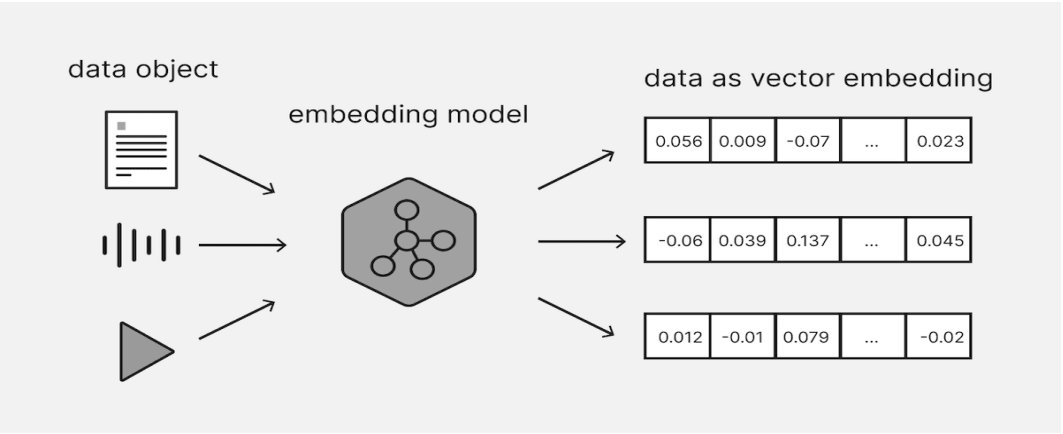


Figure: *Embedding model*

Vector database

A vector database stores, manages and indexes high-dimensional vector data. Data points are stored as arrays of numbers called “vectors,” which are clustered based on similarity.
Unlike traditional relational databases with rows and columns, data points in a vector database are represented by ‘vectors’ with a fixed number of dimensions. Because they use high-dimensional vector embeddings, vector databases are better able to handle unstructured datasets. These enable the idea of similarity search.
Vector databases work by using multiple algorithms to conduct an approximate nearest neighbor (ANN) search. The algorithms are then gathered in a pipeline to quickly and accurately retrieve and deliver data neighboring the vector that is queried.

Common pipeline for vector database:

1. **Indexing**: Indexing of vectors is conducted using various algorithms (PQ, LSH, HNSW). This step maps the vectors to a data structure that will enable faster searching.
2. **Querying**: The vector database compares the indexed query vector to the indexed vectors in the dataset to find the nearest neighbours.
3. **Post processing**: After retrieving the nearest neighbours from the dataset, it is post processed before returning to the final results. This includes steps like re-ranking the retrieved nearest neighbours.

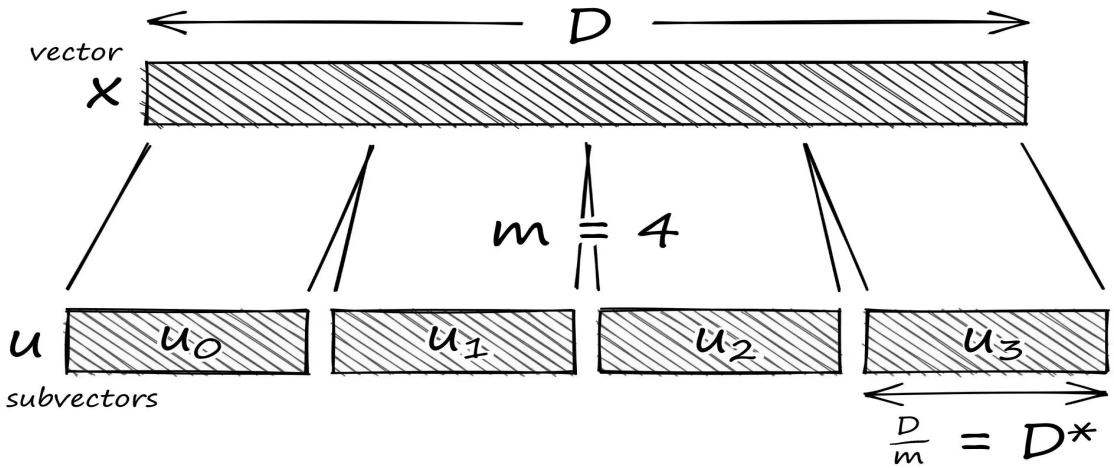
Indexing methods:

1. **Product quantization (PQ)**: It is a popular method for compressing high-dimensional vectors to use 97% less

memory, and for making nearest-neighbour search speeds 5.5x faster in our tests.

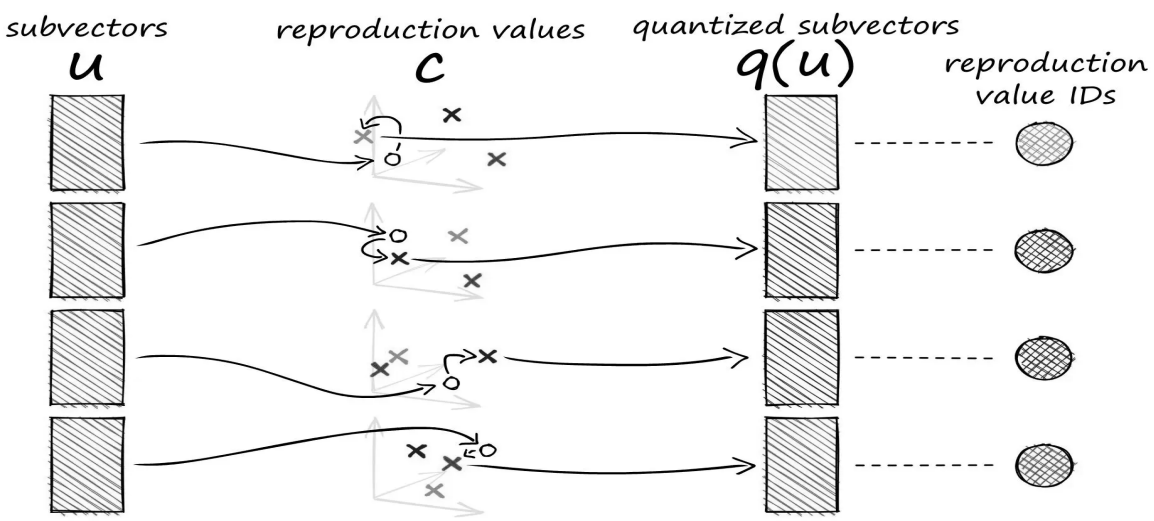
It includes:

a. *Taking a big, high dimensional vector and splitting it into equally sized chunks or sub-vectors.*



b. *Assigning each of these chunks to its nearest centroid.* Each subvector space is assigned its own set of clusters and so what we produce is a set of clustering algorithms across multiple subspaces.

c. *Assigning unique IDs to these centroids.*In PQ terminology these centroids are called <u>reproduction values</u> and are represented by $c_{j,i}$ where j is our subvector identifier, and i identifies the chosen centroid .



2. **Hierarchical Navigable Small World (HNSW):** it is currently the most popular and efficient algorithm for vector indexing. It is known for providing the most accurate answers in milliseconds.
   Fundamental concepts:

- **The hierarchy (Skip list):** In a standard linked list, to find the number, you might have to start from 1 reaching the number after many iterations.
  A Skip List adds layers of "express lanes" that skip many numbers at once.

➔ Top Layer : You move very fast across huge distances (low resolution).
➔ Middle Layer : You get closer to your number (medium resolution).
➔ Bottom Layer: You find the exact number (high resolution).

- **The search algorithm (Greedy routing):** The search process is a "Zoom-In" operation.

➔ Let q be the query vector (the user's question).
➔ Enter at the Top Layer: Start at a fixed entry point node.
➔ Greedy Step: Look at the node's neighbors. Calculate the distance from q to each neighbor. If a neighbor is closer to q than the current node, move to it. If no neighbor is closer, you have found a local minimum in this layer.
➔ Descend: Drop down to the next layer below.
➔ Repeat: Repeat the greedy step, but now with more neighbors available (finer granularity).
➔ Final Layer (Layer 0): The local minimum here is your final answer (Approximate Nearest Neighbor).

Query processing pipeline:

Query embeddings and vector similarity searches

The user query has to be embedded(converted from text to a form computers can understand) in order for it to be worked upon.

As elaborated before, **query embedding** is a mapping

$$f: Q \rightarrow R^d$$

such that a query is converted into a **dense vector** of fixed dimension d, such that semantic similarity(similar meaning) causes their corresponding vectors to be closer together.

TF-IDF:

For a word t in the query/document(d) to be embedded;
TF:Term frequency(Number of occurrences of the term/Total no. of terms)
IDF:Inverse document frequency

$$\text{IDF} = log(\frac{Total\ number\ of\ documents}{Number\ of\ documents\ containing\ the\ word})$$

Multiplied together for each term, this represents the relative weight of a term in a query (compared to others)

$$v_d = [TF(t_1, d) * IDF(t_1, d), TF(t_2, d) * IDF(t_2, d), \ldots TF(t_k, d) * IDF(t_k, d)]$$

Limitation: The context of the query, which depends on order of words and various other factors is not captured properly.

- **Word2Vec**(embedding model to express words as vectors in a continuous vector space)**:**

Distributional hypothesis:Words appearing in similar context have similar meanings

‘ *A word is characterised by the company it keeps*’

This model predicts the surrounding context words within a specific window given the current word. The input layer contains the current word and the output layer contains the context words. The hidden layer contains the number of dimensions in which we want to represent the current word present at the input layer

This is done by creating a probability distribution, the formula used for which is given below

$$P(v_w/v_o) = \left(\frac{e^{[v_w^T][v_o]}}{\Sigma e^{[v_w^T][v_o]}}\right)$$

Finding max[log(P)] gives us the exact distribution.

From word embeddings to query embedding:

$$q = \frac{1}{n}\Sigma v_{w_i}$$

However, this is not quite helpful as the order of words, which contributes to the context of the query is not considered. Example:*I like a not b* and *I like b not a* will have the same representation, which is obviously an issue

- **BERT (Bi-directional Encoder Representation from Transformers)**

  First, the given vocabulary(query/paragraph) is broken into *"tokens"* by WordPiece tokenization. It works as follows:
  1. Breaks every word into recognizable subwords, but adds ## to every broken subword to show that it is formed by breaking
  2. Starts from the beginning of each word, tries to make the longest possible chain of subwords by joining them if it recognises the joint word
  3. Continue till every word is processed
  4. Every chain is now given a token

  Each input token embedding consists of three components: token(about the subword), position(subwords' exact position) and segment(the part of sentence/query it is in)

  Now, we need to consider that a words' meaning depends on the surrounding words' meanings. By **self-attention** mechanism, every token looks at other tokens, decides which ones are relevant and the tokens' meaning is refined. This process is bi-directional, as words on both the sides can be relevant.Instead of computing attention once, BERT computes it parallely many times using multiple **'attention heads'**, each one specialised in capturing different linguistic relationships.

  After gathering contextual information, each token representation is passed through a neural network to refine its features, introduce non-linearity and allow abstract patterns to emerge.
  BERT stack many such transformer layers to progressively capture deeper abstraction(simply,meaning)

Similarity Searches:

Aims to find the most 'similar' vector to the query. Similarity is related by proximity in the vector database of embeddings. Various types of similarity metrics are given below:

- **Cosine Similarity**
  Measures the cosine of the angle between 2 vectors. In this way, it focuses on similarity of semantic direction(direction in the semantic vector-space) rather than magnitude

$$cos(\theta) = \frac{[v_q^T][v_d]}{|v_q||v_d|}$$

(Transpose taken to do multiplication)

1: Same meaning
0: unrelated
-1: exactly opposite meaning

- **Dot product similarity**
  Take only the dot product(numerator in cosine similarity) into account which makes for faster computation. Works when the vectors are already length-normalised.
- **Distance based**
  Euclidean : Normal geometric distance

$$\sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Manhattan:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} |x_i - y_i|$$

Context construction pipeline:

Once a set of documents or passages has been retrieved and reranked, the system must transform this unordered, heterogeneous set into something that can be fed to our model. This is basically a constrained optimization problem(constrained due to token limits of a model, optimization as we want the most relevant information)
Let

$$R = \{d_1, d_2, d_3, \ldots, d_n\}$$

be the set of retrieved items. From these, we have to create a context sequence such that the whole does not exceed the model's token limit

$$\Sigma_1^m \left| c_j \right| \leq Tmax$$

Context packaging:

Retrieval is focussed on the relevance of the information. Due to this, there will be overlaps between the retrieved documents. In packaging, we account for redundancy, information's order and attention efficiency.

Following are the types:-

1. **Concatenative packaging(simply joining):**
   Selects the retrieved top-k documents/passages and concatenates them based on their descending relevance score. This fails to consider redundancy(overlap).

2. **Hierarchical packaging:**

In a conventional attention model, every token competes for attention in the same model.

$$\text{Attention}(Q,K,V) = \text{Softmax}(QK^T/\sqrt{d})V$$

Thus, attention may get stretched more into less relevant things from the more relevant ones.
Hierarchy counters this by dividing the retrieved set into clusters of similar things like definitions, formulae, examples, etc and assigning them '**headers**'(kinda like summaries).
Headers act as attention basins(basically attracting most of the attention of the cluster towards them). This reduces cross-group interference.
This is simply **low-rank approximation**, where we convert several vectors(in our case, passages) into something smaller(a single vector in this case).

Prompt template construction

After building the context, clear instructions need to be given to our model for it to understand how to process the given information.
For this, we set a format of how the instructions, or "prompt" is built. This is in natural language(human language) given to the model just like how a user prompts it.

Decomposing

The prompt is broken down into **4** main parts-Instructions, Context, Query, Output constraints.

1. **Instructions**:These do not add or change facts, just change the way the output is generated by modifying the shape of the probability function

$$p(y|I, C, Q) = p(y|C, Q).e^{\Phi_I(y)}$$

: Φ_I is an activation bias term introduced by the instructions

The models see repetitive patterns of instructions and form internal directions in activation space that correlate with these instructions.

Clear from this, the term will not work for all instructions as they might not be very clear and previously learnt during training. Also since this is an activation bias, repeating the instructions or adding to it will cause little to no change as the required direction is already activated

2. **Context**: The actual facts from the documents, taken after packaging.

$$p(y|C, Q) \propto \Pi p(y|c_i, Q)$$

When there are conflicting contexts, hallucinations increase as the products don't amplify strongly.

3. **Delimiters**:Specific token sequences that the model learns during training, which are then used to mark the parts of our prompt(I,C..). This helps the model know exactly what each part has to be used for and increases accuracy by clearly marking the absolute facts, trying to override the model's prior training.

$$p(y|C) = \lambda p(y|prior) + (1 - \lambda)p(y|C)$$

Delimiters try to push the value of lambda down.

4. **Output constraints**: These just restrict the domain of responses(y) without changing the probability distribution for the ones within the allowed domain.

Generation pipeline:

The generation pipeline is the final component in a Retrieval-Augmented Generation (RAG) model, in which the Large Language Model (LLM) constructs a coherent and reliable response based on the user query and the retrieved information from the context.

Let:

- q denote the user query,
- C= $\{d_1, d_2,..., d_k\}$ denote the constructed context obtained from retrieval and context packaging,
- y= $(y_1, y_2,..., y_T)$ denote the generated output sequence which is broken into tokens.

$$y^* = \arg\max_y P_\theta(y \mid q, C)$$

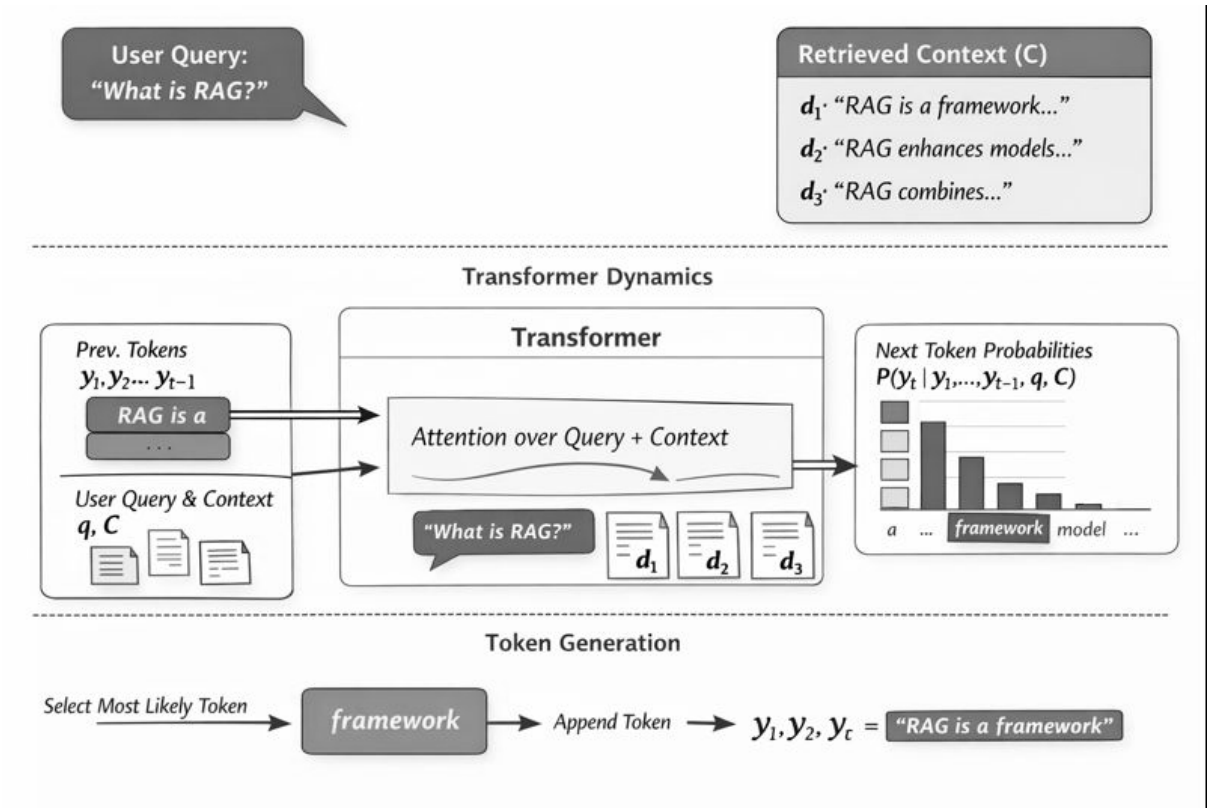
Language models do not generate full sentences at once. They generate one token at a time. That's where the chain rule of probability comes in.

Using the chain rule of probability:

$$P_\theta(y \mid q, C) = \prod_{t=1}^{T} P_\theta(y_t \mid y_{<t}, q, C)$$

where,

- $y_{<t} = (y_1, y_2, \ldots y_{t-1})$ i.e. all tokens generated before step *t*.
- $\boldsymbol{\Theta}$ represents the parameters of the LLM.
- Here, at every step the transformer attends over previous tokens, query and context and subsequently picks the next token.



Post-processing:

The post-processing stage acts as a quality control and refinement layer for the generated response. Post-processing refines the raw LLM output into a reliable, usable response. Effective post-processing can significantly improve the accuracy and coherence of the final generated response.

1. **Formatting and Structuring:** The raw text response is often formatted for better user experience. This can involve converting

the text into a desired structure like bullet points, numbered lists, paragraphs, or even a specific format such as JSON for API use. This step is deterministic and rule-based, independent of the LLM.

2. **Citation and source attribution:** A key part of the RAG technique is making sure the AI's answer comes directly from the information it was given. This "grounding" prevents the AI from making up false information or "hallucinating".
   Post-processing can involve automatically adding citations or references (e.g., "[Doc 1]", "[Source: *Employee Handbook*]") that link specific parts of the answer back to the source documents used, allowing users to verify the information.
   RAG does not explicitly label sources during generation. In order to add citations, explainability and trust, we must add a separate attribution step. This is known as **post-hoc attribution**.

   Post-hoc attribution: Post-hoc attribution is a technique in RAG where citations are assigned to generated answer segments after generation by aligning them with retrieved document chunks using embedding similarity.

   The generated answer is not treated as one block. It is instead split into “answer spans”. Answer span can be a sentence, a clause or even a bullet point.Thus,we can define:

$$A = \{a_1, a_2, \ldots, a_m\}$$

   Constructed context as previously defined can be written as:

$$C = \{d_1, d_2, \ldots, d_k\}$$

   Now, we embed answer spans and context chunks using the same embedding model such that everything lies in the same semantic space.

$$f(\cdot) : \text{text} \rightarrow \mathbb{R}^n$$

Finally, answer spans are matched with document chunks and the most semantically similar chunk becomes the citation.

$$\text{cite}(a_i) = \arg\max_{d_j \in C} \text{sim}(f(a_i), f(d_j))$$

3. **Confidence scoring:** Confidence scoring in RAG quantifies answer reliability using signals such as average retrieval similarity, token-level log-likelihood, or LLM-based self-evaluation, enabling risk-aware and human-in-the-loop systems.
   Confidence scores are required to warn users about the risk when the confidence scores are below a certain threshold value. If the confidence score is too low, automatic rejection or fallback occurs.

   - Mean retrieval similarity: This method compares the final output with the user query using similarity scores such as cosine similarity etc. High average similarity suggests that the retrieval process was good and the output is well-grounded.

$$\text{Confidence} = \frac{1}{k} \sum_{i=1}^{k} \text{sim}(q, d_i)$$

   - Log-likelihood of generated tokens: This mechanism operates based on the probabilistic selection of tokens during their generation process. Higher probability of tokens suggests that the model is confident in its output.

$$\text{Confidence} = \frac{1}{T} \sum_{t=1}^{T} \log P(y_t \mid y_{<t}, q, C)$$

- LLM-based self evaluation: In this method, the LLM itself acts as a judge to predict reliability of its own answer. We can rely on this since LLMs are good at consistency checking, identifying unsupported claims, and detecting missing evidence. LLM usually gives this output as a score or a label (high/medium/low).

4. **The final safety check:** The final safety check involves applying post-generation classifiers to detect content that is harmful, biased, or violates established policies. The output is accepted only if its risk score is below a predefined threshold; otherwise, it triggers regeneration, refusal, or human review.
   This checks for toxicity (hate speech, harassment, abusive language, threats, etc.), bias (gender, racial, religious, etc.), policy violation checks etc.

   We can define a toxicity scoring function **T(y)**:

   $$T(y) \in [0, 1]$$

   - 0 → completely safe
   - 1 → highly toxic

   Similarly, we define a safety threshold τ .

   Acceptance function:

   $$\text{Accept}(y) = \begin{cases} 1 & \text{if } T(y) < \tau \\ 0 & \text{otherwise} \end{cases}$$

Evaluation and improvement:

Evaluation is the most critical component of a RAG system. Unlike classical ML systems where accuracy can be directly measured, RAG systems are multi-stage pipelines, and errors

can originate from retrieval, context construction, generation or system design. Hence, evaluation must be modular and layered.

Retrieval Evaluation:

Retrieval evaluation measures how well the retriever fetches relevant documents for a given query (independent of generation). This is done with the help of various functions.

Notations:

G: set of ground-truth relevant documents

A_k: set of top-k retrieved documents

Recall: Ratio of the documents that were retrieved to the total documents that were supposed to be retrieved. Low recall suggests missing knowledge leading to imperfect answers. RAG systems prioritize recall over precision in early stages.

$$\text{Recall@K} = \frac{|A_K \cap G|}{|G|}$$

Precision: Ratio of relevant documents to the total retrieved documents. It can be improved later using re-ranking.

$$\text{Precision@K} = \frac{|A_K \cap G|}{K}$$

Retrieval improvement:

Once the weaknesses are identified, retrieval can be improved.

Better chunking: Choosing large chunks causes poor granularity while small chunks causes loss of context. We should therefore use effective methods of chunking such as Recursive

chunking, Semantic chunking, etc.Good chunking significantly improves Recall.

Query rewriting: This rewrites user queries to make it clear, more explicit, and better aligning with document language. This improves embedding quality and reduces ambiguity.

Hybrid retrieval: This combines sparse retrieval (BM25) and dense retrieval (vector search) to improve keyword precision and semantic similarity simultaneously.

Generative evaluation:

This measures the quality of the generated answer by focusing on semantic properties.

Faithfulness: Faithfulness measures whether the generated answer is fully supported by the retrieved context.

Context relevance: It measures how relevant the retrieved context is to the generated answer.

Answer correctness: It measures whether the answer is factually and semantically correct.

System optimisation: The system needs to be fast, scalable, and affordable. This can be achieved through caching, Latency reduction, Cost-aware retrieval, Adaptive pipelines, etc.

Applications and Advantages:

Reduced hallucinations

Grounding responses in retrieved documents significantly lowers the probability of fabricated or unsupported statements.

Improved factual accuracy
The generation process is constrained by real, retrieved information, making outputs more reliable than purely parametric models.

Up-to-date responses
RAG separates knowledge storage from the model itself, allowing systems to reflect new information without modifying model weights.

Source grounding and traceability
Retrieved context can be inspected or cited, improving transparency and trust in the generated output.

Efficient domain adaptation
The same language model can be reused across domains by changing only the retrieval corpus, reducing computational and operational cost.

Scalability
RAG scales effectively to large knowledge bases using vector search, enabling reasoning over datasets far larger than what a model could memorize.

Code implementation:

```
#importing and setting up libraries
import os
import numpy as np
import faiss
import torch
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForCausalLM
from sklearn.metrics.pairwise import cosine_similarity
import nltk
```

```python
nltk.download('punkt')
from nltk.tokenize import sent_tokenize

## DATA PREPARATION PIPELINE

#document ingestion
def load_documents(folder_path):
    documents = []
    for file in os.listdir(folder_path):
        if file.endswith(".txt"):
            with open(os.path.join(folder_path, file), "r", encoding="utf-8") as f:
                documents.append(f.read())
    return documents

#chunking
def chunk_document(text, chunk_size=200, overlap=50):
    sentences = sent_tokenize(text)
    chunks = []
    current_chunk = []

    for sentence in sentences:
        current_chunk.append(sentence)
        if len(" ".join(current_chunk).split()) >= chunk_size:
            chunks.append(" ".join(current_chunk))
            current_chunk = current_chunk[-overlap:]

    if current_chunk:
        chunks.append(" ".join(current_chunk))

    return chunks

#embedding generation
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

def embed_texts(texts):
    return embedding_model.encode(texts, show_progress_bar=True)

#vector database
def build_faiss_index(embeddings):
    dimension = embeddings.shape[1]
```

```
    index = faiss.IndexFlatIP(dimension)  # Inner Product (Cosine similarity)
    faiss.normalize_L2(embeddings)
    index.add(embeddings)
    return index

## QUERY PROCESSING PIPELINE

#query embedding
def embed_query(query):
    embedding = embedding_model.encode([query])
    faiss.normalize_L2(embedding)
    return embedding

#vector similarity search
def retrieve_top_k(query_embedding, index, chunks, k=5):
    scores, indices = index.search(query_embedding, k)
    retrieved_chunks = [chunks[i] for i in indices[0]]
    return retrieved_chunks, scores[0]

#reranking
def rerank(query_embedding, retrieved_chunks, lambda_weight=0.8):
    chunk_embeddings = embed_texts(retrieved_chunks)
    similarity = cosine_similarity(query_embedding, chunk_embeddings)[0]
    length_norm = np.array([len(c.split()) for c in retrieved_chunks])
    length_norm = length_norm / length_norm.max()

    final_scores = lambda_weight * similarity + (1 - lambda_weight) * length_norm
    ranked = sorted(zip(retrieved_chunks, final_scores), key=lambda x: x[1],
      reverse=True)

    return [r[0] for r in ranked]

## CONTEXT CONSTRUCTION PIPELINE

#context packaging
def build_context(chunks, max_tokens=800):
    context = ""
    for chunk in chunks:
        if len(context.split()) + len(chunk.split()) <= max_tokens:
            context += chunk + "\n"
```

```
        else:
            break
    return context

#prompt template construction
def build_prompt(context, query):
    return f"""
You are an expert assistant. Answer ONLY using the context below.

Context:
{context}

Question:
{query}

Answer:
"""

## GENERATION PIPELINE

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

def generate_answer(prompt, max_new_tokens=200):
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            temperature=0.7,
            top_p=0.9
        )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

#post-processing
def post_process(text):
    return text.split("Answer:")[-1].strip()

## EVALUATION
```

```python
#evaluation metrics
def recall_at_k(retrieved, ground_truth):
    return len(set(retrieved) & set(ground_truth)) / len(ground_truth)

def precision_at_k(retrieved, ground_truth):
    return len(set(retrieved) & set(ground_truth)) / len(retrieved)


# FULL RAG PIPELINE
def rag_pipeline(query, index, chunks):
    q_embed = embed_query(query)
    retrieved, _ = retrieve_top_k(q_embed, index, chunks, k=8)
    reranked = rerank(q_embed, retrieved)
    context = build_context(reranked)
    prompt = build_prompt(context, query)
    raw_answer = generate_answer(prompt)
    return post_process(raw_answer)
```

References:

https://www.pinecone.io/learn/retrieval-augmented-generation/
https://writingmate.ai/blog/the-best-embedding-models
https://arxiv.org/pdf/2510.12323
https://www.pinecone.io/learn/vector-database/
https://aws.amazon.com/what-is/retrieval-augmented-generation/
https://www.ibm.com/think/topics/retrieval-augmented-generation